**TUGAS 04 – IMPLEMENTASI UNIT TESTING DENGAN GITHUB ACTIONS**

Nama : Muhammad Mukhlis Rifdulloh

NIM : 3224014

Kelas : Sistem Informasi

Mata Kuliah : Implementasi Pengujian PL

Tanggal : 13 Oktober 2026

---

## BAGIAN A – GIT & GITHUB DASAR

### SOAL 1 KONSEP GIT DAN VERSION CONTROL

#### 1. Perbedaan Centralized VCS dan Distributed VCS

Centralized Version Control System (Centralized VCS) adalah sistem pengelolaan versi yang menggunakan satu server pusat sebagai tempat penyimpanan utama seluruh riwayat perubahan file. Pengguna perlu terhubung ke server tersebut untuk melakukan banyak aktivitas pengelolaan versi. Contohnya adalah **Apache Subversion (SVN)** dan **CVS (Concurrent Versions System)**.

Distributed Version Control System (Distributed VCS) adalah sistem pengelolaan versi yang memungkinkan setiap pengguna memiliki salinan repository beserta riwayat perubahannya di komputer masing-masing. Pengguna dapat melakukan commit secara lokal sebelum mengirimkan perubahan ke repository bersama. Contohnya adalah **Git** dan **Mercurial**.

Perbedaan utamanya terletak pada penyimpanan riwayat versi. Centralized VCS bergantung pada server pusat, sedangkan Distributed VCS memiliki salinan riwayat repository pada setiap komputer yang melakukan clone.

#### 2. Lima Git commands yang wajib dikuasai

| No | Git command | Fungsi |
|---|---|---|
| 1 | git init | Membuat repository Git baru di direktori lokal. |
| 2 | git clone | Menyalin repository dari server ke komputer lokal. |

| | | |
|---|---|---|
| 3 | git add | Memasukkan perubahan file ke staging area sebelum commit. |
| 4 | git commit | Menyimpan perubahan ke dalam riwayat repository lokal. |
| 5 | git push | Mengirimkan commit local ke repository remote. |

### 3. Fungsi file .gitignore

File .gitignore digunakan untuk menentukan file atau folder yang tidak perlu dilacak dan dimasukkan ke dalam repository Git. File ini berguna untuk menghindari pengunggahan file sementara, hasil kompilasi, konfigurasi lokal, atau informasi sensitif seperti kredensial.

Contoh isi .gitignore:

- *.log
- .env
- target/
- .idea/

Dengan aturan tersebut, file berakhiran .log, file .env, folder target, dan folder .idea akan diabaikan oleh Git apabila belum terlanjur dilacak. File .gitignore tidak otomatis menghapus file yang sebelumnya sudah masuk ke dalam tracking Git.

## SOAL 2 PRAKTIK GIT COMMANDS

Berikut urutan command untuk membuat repository, menambahkan README, membuat branch, melakukan commit, dan menggabungkan branch ke main.

### 1. Clone repository ke computer local

```
git clone https://github.com/mkhlisrr/latihan-git-3224014.git

cd latihan-git-3224014
```

### 2. Membuat file README.md

Pada terminal Git Bash atau terminal VS Code, jalankan:

```
echo "# Muhammad Mukhlis Rifdulloh - 3224014" > README.md

git add README.md

git commit -m "docs: add README"
```

```
git push -u origin main
```

Command tersebut membuat README, memasukannya ke staging area, menyimpan commit, lalu mengirimkannya ke branch main pada GitHub.

3. **Membuat dan berpindah ke branch feature-test**

```
git switch -c feature-test
```

```
git switch feature-test
```

Alternatifnya, kedua Langkah tersebut dapat dilakukan sekaligus dengan:

```
git switch -c feature-test
```

Gunakan salah satu cara saja.

4. **Membuat file test.txt dan melakukan commit**

```
echo "File untuk latihan unit testing" > test.txt
```

```
git add test.txt
```

```
git commit -m "test: add test file"
```

5. **Push branch feature-test ke GitHub**

```
git push -u origin feature-test
```

Setelah command tersebut berhasil, branch feature-test akan tersedia pada repository remote.

6. **Merge feature-test ke main.**

```
git switch main
```

```
git pull origin main
```

```
git merge feature-test
```

```
git push origin main
```

Command pertama berpindah kembali ke main. Command kedua mengambil pembaruan terbaru dari GitHub. Selanjutnya, perubahan pada feature-test digabungkan ke main, kemudian hasil penggabungan dikirim ke GitHub.

Apabila repository menggunakan aturan perlindungan branch, penggabungan dapat perlu dilakukan melalui Pull Request di GitHub.

**BAGIAN B – UNIT TESTING FRAMEWORK**

**SOAL 3 KONVERSI TEST CASE KE KODE UNIT TEST**

1. **Lima test case berdasarkan independent path**

Program validasiPendaftaran() memiliki beberapa kondisi yang diperiksa secara berurutan. Untuk menguji jalur independennya, setiap test case dibuat dengan memicu kondisi tertentu, sedangkan kondisi sebelumnya dibuat valid agar pengujian mencapai bagian yang dituju.

| No | Nama test case | Input nama | Umur | Email | Setuju syarat | Hasil yang diharapkan |
|---|---|---|---|---|---|---|
| 1 | Nama kosong | “” | 20 | mukhlis@gmail.com | true | Nama harus diisi |
| 2 | Umur kurang dari 17 | Mukhlis | 16 | mukhlis@gmail.com | true | Umur minimal 17 tahun |
| 3 | Email tidak valid | Mukhlis | 20 | mukhlisgmail.com | true | Email tidak valid |
| 4 | Tidak menyetujui syarat | Mukhlis | 20 | mukhlis@gmail.com | false | Harus menyetujui syarat dan ketentuan |
| 5 | Semua input valid | Mukhlis | 20 | mukhlis@gmail.com | true | Pendaftaran berhasil |

Kelima test case tersebut mencakup empat kondisi gagal dan satu kondisi berhasil. Dengan demikian, setiap jalur utama pada logika validasi dapat diuji.

2. **Implementasi unit test menggunakan Junit**

Contoh berikut menggunakan **JUnit 5**. Misalkan method `validasiPendaftaran()` berada di dalam class `PendaftaranService`.

File `PendaftaranServiceTest.java`:

```
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
```

```
public class PendaftaranServiceTest {

    private final PendaftaranService service =
            new PendaftaranService();

    @Test
    void testNamaKosong() {
        String hasil = service.validasiPendaftaran(
                "", 20, "andi@gmail.com", true
        );

        assertEquals("Nama harus diisi", hasil);
    }

    @Test
    void testUmurKurangDari17() {
        String hasil = service.validasiPendaftaran(
                "Andi", 16, "andi@gmail.com", true
        );

        assertEquals("Umur minimal 17 tahun", hasil);
    }

    @Test
    void testEmailTidakValid() {
        String hasil = service.validasiPendaftaran(
```

```
                "Andi", 20, "andigmail.com", true
        );

        assertEquals("Email tidak valid", hasil);
    }

    @Test
    void testTidakMenyetujuiSyarat() {
        String hasil = service.validasiPendaftaran(
                "Andi", 20, "andi@gmail.com", false
        );

        assertEquals(
                "Harus menyetujui syarat dan ketentuan",
                hasil
        );
    }

    @Test
    void testPendaftaranBerhasil() {
        String hasil = service.validasiPendaftaran(
                "Andi", 20, "andi@gmail.com", true
        );

        assertEquals("Pendaftaran berhasil", hasil);
    }
}
```

**Catatan implementasi:** Class PendaftaranService harus tersedia di project dan berisi method validasiPendaftaran() sesuai soal. Jika nama class atau package pada project berbeda, sesuaikan deklarasi dan import-nya. Pengujian dijalankan menggunakan JUnit 5 yang sudah ditambahkan sebagai dependency project.

**SOAL 4 IMPLEMENTASI UNIT TEST UNTUK CLASS STUDENTGRADE**

1. **Menghitung Cyclomatic Complexity method calculateGrade()**

Cyclomatic Complexity (CC) merupakan ukuran kompleksitas logika suatu program berdasarkan jumlah jalur independen yang dapat dilalui.

Pada method `calculateGrade(),` terdapat lima struktur keputusan utama:

- Pemeriksaan nilai di luar rentang 0–100.
- Pemeriksaan nilai minimal 80.
- Pemeriksaan nilai minimal 70.
- Pemeriksaan nilai minimal 60.
- Pemeriksaan nilai minimal 50.

Dengan rumus:

**CC = jumlah decision + 1**

Maka:

CC = 5 + 1 = **6**

Jadi, berdasarkan penghitungan setiap struktur keputusan if dan else if sebagai satu decision, Cyclomatic Complexity method calculateGrade() adalah **6**. Penghitungan ini menggunakan pendekatan keputusan pada struktur kontrol; jika operator logika || juga dihitung sebagai kondisi terpisah, hasil dapat berbeda sesuai konvensi yang digunakan.

2. **Independent Path method calculateGrade()**

Berdasarkan penghitungan CC di atas, terdapat enam jalur independen utama:

| Jalur | Kondisi | Contoh input | Hasil |
|---|---|---|---|
| P1 | Nilai kurang dari 0 | -1 | IllegalArgumentException |
| P2 | Nilai 80 sampai 100 | 85 | A |
| P3 | Nilai 70 sampai 79 | 75 | B |

| | | | |
|---|---|---|---|
| P4 | Nilai 60 sampai 69 | 65 | C |
| P5 | Nilai 50 sampai 59 | 55 | D |
| P6 | Nilai 0 sampai 49 | 40 | E |

Pemeriksaan nilai lebih dari 100 juga perlu diuji untuk memastikan validasi batas atas bekerja dengan benar, walaupun tabel jalur utama di atas hanya mencantumkan satu contoh nilai yang tidak valid.

### 3. Implementasi unit test JUnit untuk kedua method

Misalkan class StudentGrade berada di package yang sama dengan file pengujian. File berikut menggunakan JUnit 5.

File `StudentGradeTest.java`:

```
import org.junit.jupiter.api.Test;


import static org.junit.jupiter.api.Assertions.*;


public class StudentGradeTest {


    private final StudentGrade studentGrade =
            new StudentGrade();


    @Test
    void testGradeA() {
        assertEquals("A", studentGrade.calculateGrade(85));
    }


    @Test
    void testGradeB() {
        assertEquals("B", studentGrade.calculateGrade(75));
    }
```

```
@Test
void testGradeC() {
    assertEquals("C", studentGrade.calculateGrade(65));
}

@Test
void testGradeD() {
    assertEquals("D", studentGrade.calculateGrade(55));
}

@Test
void testGradeE() {
    assertEquals("E", studentGrade.calculateGrade(40));
}

@Test
void testScoreDiBawahNol() {
    assertThrows(
            IllegalArgumentException.class,
            () -> studentGrade.calculateGrade(-1)
    );
}

@Test
void testScoreDiAtasSeratus() {
    assertThrows(
```

```
                IllegalArgumentException.class,
                () -> studentGrade.calculateGrade(101)
        );
    }

    @Test
    void testLulus() {
        assertTrue(studentGrade.isPassed(60));
    }

    @Test
    void testTidakLulus() {
        assertFalse(studentGrade.isPassed(59));
    }

    @Test
    void testIsPassedScoreTidakValid() {
        assertThrows(
                IllegalArgumentException.class,
                () -> studentGrade.isPassed(101)
        );
    }
}
```

Pengujian tersebut memeriksa seluruh kategori nilai, validasi nilai di luar rentang yang diperbolehkan, serta kedua hasil dari method isPassed(). Nilai 60 diuji sebagai batas minimal lulus, sedangkan nilai 59 diuji sebagai batas nilai yang belum lulus.

**BAGIAN C – GITHUB ACTIONS**

**SOAL 5 Analisis Workflow YAML**

1. **Nama Workflow**

Nama workflow pada YAML yang diberikan adalah **Python CI,** sesuai deklarasi

```
name: Python CI
```

2. **Trigger atau event yang memicu workflow**

Workflow dijalankan oleh dua jenis event:

- **Push** ke branch main atau develop.
- **Pull request** yang ditujukan ke branch main.

Dengan konfigurasi tersebut, proses pengujian otomatis berjalan ketika ada perubahan yang dikirim ke kedua branch tersebut atau ketika ada Pull Request menuju main.

3. **Fungsi strategy matrix**

Strategy matrix digunakan untuk menjalankan job yang sama pada beberapa variasi konfigurasi. Pada workflow tersebut, pengujian dijalankan menggunakan Python versi 3.10, 3.11, dan 3.12.

Keuntungannya adalah pengembang dapat mengetahui apakah program tetap berjalan dengan baik pada beberapa versi Python tanpa perlu membuat workflow terpisah untuk setiap versi. GitHub Actions akan membuat variasi job berdasarkan versi yang tercantum di dalam matrix.

4. **Steps yang terdapat di dalam job test**

Pada YAML yang diberikan, terdapat **empat steps eksplisit**:

1. **Checkout code** — menggunakan actions/checkout@v4 untuk mengambil kode repository ke runner.
2. **Setup Python** — menggunakan actions/setup-python@v5 untuk memasang versi Python sesuai matrix.
3. **Install dependencies** — memperbarui pip serta memasang pytest dan pytest-cov.
4. **Run tests** — menjalankan pytest --cov=src --cov-report=xml untuk menguji program dan menghasilkan laporan coverage dalam format XML.

**SOAL 6 MEMBUAT WORKFLOW YAML UNTUK JAVA**

Berikut contoh file workflow lengkap sesuai ketentuan soal. File disimpan dengan nama .github/workflows/java-unit-test.yml.

```
name: Java Unit Test


on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]


jobs:
  test:
    runs-on: ubuntu-latest


    steps:
      - name: Checkout code
        uses: actions/checkout@v4


      - name: Setup JDK 17
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '17'
          cache: maven


      - name: Run tests
```

```
    run: mvn test

- name: Generate coverage report
  run: mvn jacoco:report

- name: Upload coverage artifact
  uses: actions/upload-artifact@v4
  with:
    name: jacoco-report
    path: target/site/jacoco/
```

Workflow tersebut menggunakan runner ubuntu-latest, memasang JDK 17, menjalankan pengujian dengan Maven, menghasilkan laporan coverage, kemudian mengunggah folder laporan sebagai artifact bernama jacoco-report.

**BAGIAN D – CODE COVERAGE & INTEGRASI**

**SOAL 7 ANALISIS CODE COVERAGE**

Berdasarkan laporan coverage pada soal, diperoleh data berikut:

| File | Stmts | Miss | Coverage |
|---|---|---|---|
| src/calculator.py | 10 | 0 | 100% |
| src/login_system.py | 15 | 2 | 87% |
| src/payment.py | 20 | 8 | 60% |
| src/utils.py | 5 | 1 | 80% |
| **TOTAL** | **50** | **11** | **78%** |

1. **Total statement dalam project**

Total statement diperoleh dengan menjumlahkan seluruh statement pada setiap file:

10 + 15 + 20 + 5 = **50 statement**

2. **Total statement yang tidak tereksekusi**

Jumlah statement yang tidak tereksekusi atau Miss adalah:

0 + 2 + 8 + 1 = **11 statement**

Jadi, terdapat 11 statement yang belum dijalankan selama proses pengujian.

3. **File dengan coverage paling rendah**

File dengan coverage paling rendah adalah **src/payment.py**, dengan coverage sebesar **60%**. Dari total 20 statement, terdapat 8 statement yang belum tereksekusi.

4. **Cara meningkatkan coverage pada payment.py**

Untuk meningkatkan coverage pada payment.py, langkah pertama adalah memeriksa bagian kode yang belum dijalankan berdasarkan laporan coverage. Setelah itu, buat test case yang mencakup berbagai kemungkinan proses pembayaran.

Pengujian dapat mencakup pembayaran berhasil, saldo tidak mencukupi, jumlah pembayaran tidak valid, pembayaran dengan nilai batas tertentu, serta kondisi ketika transaksi mengalami kegagalan. Setiap test case perlu memiliki assertion untuk memastikan hasil program sesuai dengan yang diharapkan.

Selain menambah jumlah test case, pengujian juga harus memeriksa jalur logika yang berbeda, bukan hanya menjalankan baris kode secara sembarangan. Dengan cara tersebut, peningkatan coverage dapat sekaligus membantu menemukan kesalahan pada proses pembayaran.

5. **Apakah coverage 78% sudah cukup baik?**

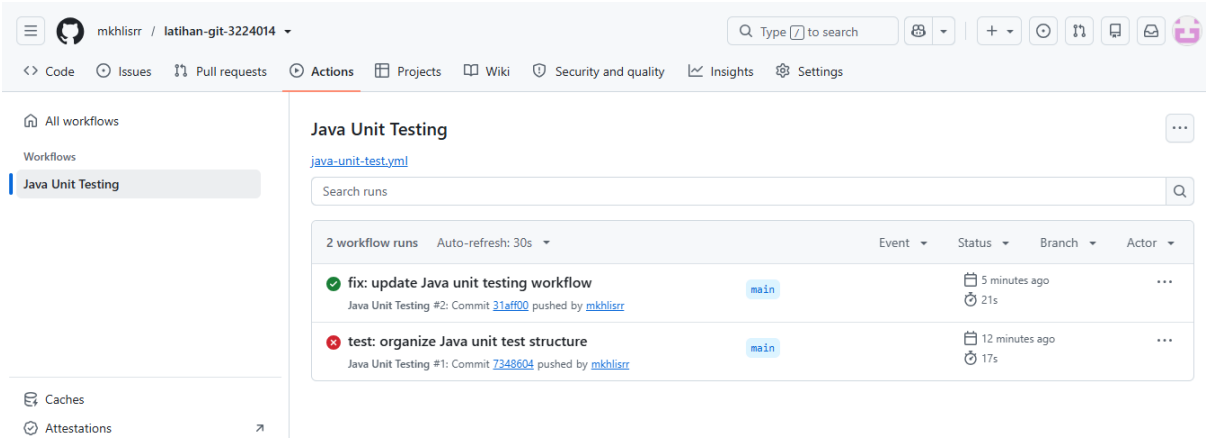
Menurut saya, coverage 78% merupakan hasil yang cukup baik sebagai gambaran awal pengujian karena sebagian besar statement telah tereksekusi. Namun, angka tersebut belum tentu menjamin program bebas dari kesalahan.

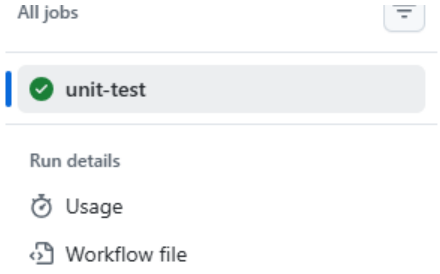
Masih terdapat 11 statement yang belum tereksekusi, terutama pada payment.py yang memiliki coverage paling rendah, yaitu 60%. Bagian pembayaran perlu mendapat perhatian lebih karena kesalahan pada proses tersebut dapat memengaruhi ketepatan transaksi.

Oleh karena itu, coverage 78% dapat dijadikan titik awal, tetapi pengujian sebaiknya dilanjutkan pada bagian yang belum tercakup dan pada kondisi-kondisi penting. Kualitas pengujian tidak cukup dinilai hanya dari persentase coverage, tetapi juga dari kemampuan test case dalam memeriksa hasil dan menemukan kesalahan.

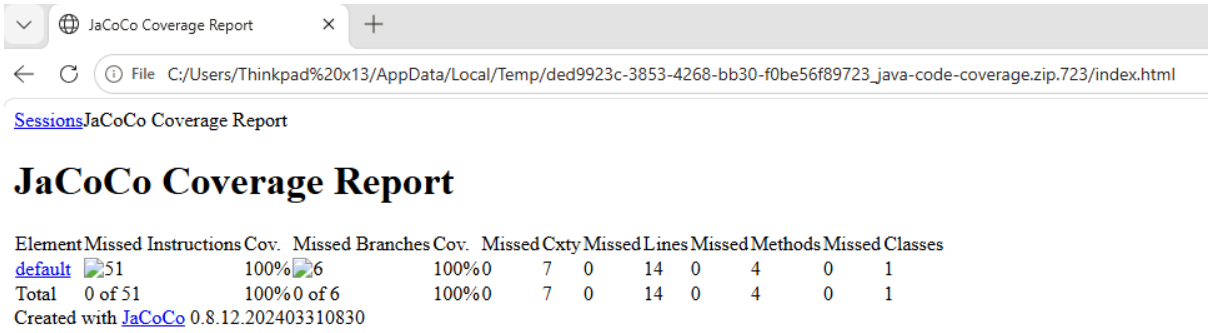
## BAGIAN E – STUDI KASUS TERPADU (BONUS)

## SOAL 8 IMPLEMENTASI LENGKAP BANKACCOUNT






Pada tugas ini, saya membuat program sederhana bernama BankAccount menggunakan Java 17. Program tersebut memiliki fungsi untuk melihat saldo, melakukan penyetoran uang (*deposit*), dan melakukan penarikan uang (*withdraw*). Saya juga menambahkan validasi untuk menolak nominal negatif dan penarikan yang melebihi saldo.

Pengujian unit dibuat menggunakan JUnit 5 dengan total 8 skenario pengujian. Skenario tersebut mencakup pemeriksaan saldo awal, penyetoran berhasil, penarikan berhasil, penyetoran dan penarikan dengan nominal negatif, penarikan melebihi saldo, penyetoran nol, serta penarikan seluruh saldo.

Untuk mengotomatisasi proses pengujian, saya menggunakan GitHub Actions melalui file workflow java-unit-test.yml. Setiap kali perubahan dikirim ke branch main atau feature-test, GitHub Actions akan menyiapkan Java 17, mengunduh library yang dibutuhkan, mengompilasi program dan unit test, menjalankan pengujian, serta membuat laporan code coverage menggunakan JaCoCo. Laporan tersebut kemudian diunggah sebagai artifact agar dapat diunduh dan diperiksa.

Berdasarkan hasil implementasi, seluruh 8 unit test berhasil dijalankan dan workflow GitHub Actions menunjukkan status berhasil. Laporan JaCoCo menunjukkan instruction coverage sebesar 100% (51 dari 51 instruksi) dan branch coverage sebesar 100% (6 dari 6 branch). Hasil ini menunjukkan bahwa seluruh instruksi dan percabangan yang terukur dalam program telah tercakup oleh skenario pengujian yang dibuat.

Dari praktik ini, saya memahami bahwa unit testing membantu memeriksa fungsi program secara terpisah, sedangkan GitHub Actions memudahkan proses pengujian otomatis setiap kali ada perubahan kode. Dengan demikian, kesalahan dapat diketahui lebih awal dan kualitas kode lebih mudah dipantau.